

FreeRTOS 快速开发文档

1 添加自己的代码

进入目录 `freertos/vendor`，对文件 `vendor.c` 操作如下图

```
1  #include <stdio.h>
2
3  void vendor_init(void)
4  {
5      printf("vendor init...\n");
6      printf("hello world..\n");
7  }
8
```

编译烧录后显示如下图

```
J-Boot SPL 2013.07-00006-gc91964e (Oct 11 2019 - 19:31:16)
ERROR EPC 81f64644
CPM_CPAPCR:a9008520
CPM_CPMPCR:98000083
CPM_CPCCR:9a052210
GD25Q256CYIG 00c84019
[0.0000] xburst rtos @ Dec 26 2019 09:28:08, epc: 81f64644
[0.4083] vendor init...
[0.4104] hello world..
```

2 添加源文件,加入通用编译流程

支持格式: `.c .s .S .cc .cpp` 目前暂不支持C++

以在 `freertos/vendor` 目录下添加文件 `add.c`

在 `freertod/vendor/Makefile` 中添加

```
1 | src-y += add.c
```

加入后编译如下所示

```
LD      newlib/builtin.o
CC      symbols/func_symbols.c
LD      dfs/builtin.o
CC      vendor/add.c
CC      vendor/vendor.c
LD      drivers/builtin.o
LD      shell/builtin.o
LD      symbols/builtin.o
LD      dfs/debugfs/builtin.o
LD      os/freertos/builtin.o
LD      vendor/builtin.o
LD      dfs/mtd/builtin.o
```

在 `vendor.c` 中引用 `add.c` 中的函数

```
1  #include <stdio.h>
2  #include "add.h"
3  void vendor_init(void)
4  {
5      printf("vendor init...\n");
6      printf("hello world..\n");
7      add(2,3);
8  }
9
```

编译烧录结果如下

```
ERROR EPC 81f64644
CPM_CPAPCR:a9008520
CPM_CPMPCR:98000083
CPM_CPCCR:9a052210
GD25Q256CYIG 00c84019
[0.0000] xburst rtos @ Dec 26 2019 09:28:08, epc: 81f64644
[0.4088] vendor init...
[0.4109] hello world..
[0.4129] 2 + 3 = 5
```

3 在外部编译静态库文件

生成.o文件

```
1 | tools/toolchains/mips-gcc520-elf/bin/mips-sde-elf-gcc -c -G 0 xxx.c
```

特别注意：在编译时必须加上 `-G 0` 参数，否则制作成静态库后，使用会报错！

```
mips-linux-gnu-ld: small-data section exceeds 64KB; lower small-data size limit (see option -G)
```

```
package/target_rule.mk:29: recipe for target 'target_default' failed
make: *** [target_default] Error 1
```

生成.a文件

```
1 | tools/toolchains/mips-gcc520-elf/bin/mips-sde-elf-ar -rcs libmul.a xxx.o
```

4 添加目标文件,加入通用编译流程

支持格式: `.o` `.a`

以在 `freertos/vendor` 中添加 `libmul.a`

在 `freertos/package/vendor/vendor.mk` 中添加

```
1 | package_lib-y += vendor/libmul.a
```

在 `vendor.c` 引用 `libmul.a` 中的函数

```
1  #include <stdio.h>
2  #include "add.h"
3  #include "mul.h"
4  void vendor_init(void)
5  {
6      printf("vendor init...\n");
7      printf("hello world..\n");
8      add(2,3);
9      printf("mul = %d\n",mul(3,4));
10 }
11 |
```

烧录结果显示

```
CPM_CPAPCR:a9008520
CPM_CPMPCR:98000083
CPM_CPCCR:9a052210
GD25Q256CYIG 00c84019
[0.0000] xburst rtos @ Dec 26 2019 09:28:08, epc: 81f64644
[0.4098] vendor init...
[0.4119] hello world..
[0.4140] 2 + 3 = 5
[0.4157] mul = 12
```

5 添加自动检索头文件目录

添加自动检索头文件目录: 可以用 `#include <xxx.h>` 添加头文件, 操作如下:

对目录 `freertos/package/vendor` 下的 `vendor.mk` 文件代码添加include头文件目录

```
1 | CFLAGS += -Ivendor/include
```

6 添加宏控可在IconfigTool中修改

进入 `package/vendor/` 目录对 `Config.in` 操作

如下图例子

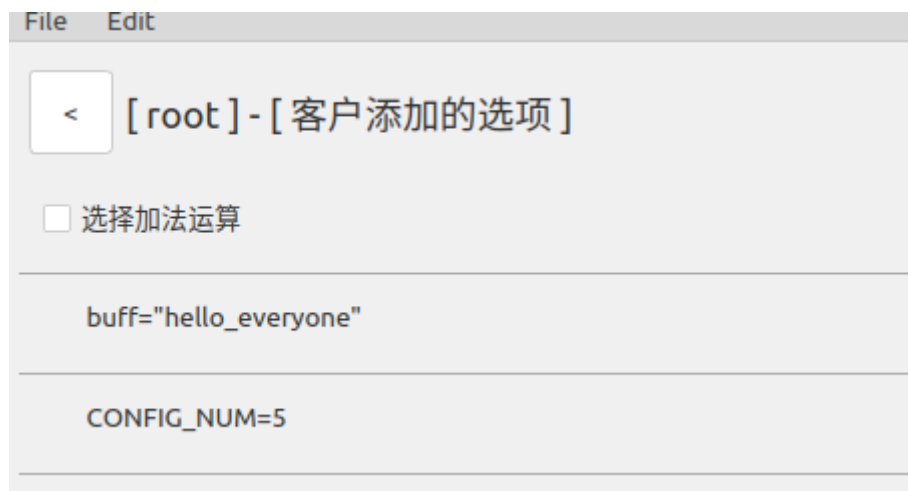
```
package / vendor / = Config.in
1  menu "客户添加的选项"
2
3  config CONFIG_VENDOR_ADD
4  ... bool "选择加法运算"
5  ... default n ← 默认不选择
6
7  config CONFIG_VENDOR_STRING
8  ... string "buff"
9  ... default "hello_everyone"
10
11 config CONFIG_VENDOR_NUM
12 ... int "CONFIG_NUM"
13 ... default 5
14
15 endmenu
16

void vendor_init(void)
{
... int b = CONFIG_VENDOR_NUM;
... char buf[20] = CONFIG_VENDOR_STRING;
... printf("vendor init...\n");
... printf("hello world..\n");
#ifdef CONFIG_VENDOR_ADD
... add(2,3);
#else
... printf("mul = %d\n",mul(3,4));
#endif
... printf("CONFIG_VENDOR_NUM = %d\n",b);
... printf("CONFIG_VENDOR_STRING = %s\n",buf);
}
```

烧录后的结果如下图

```
U-Boot SPL 2013.07-00006-gc91964e (Oct 11 2019 - 19:31:16)
ERROR EPC 81f64644
CPM_CPAPCR:a9008520
CPM_CPMPCTR:98000083
CPM_CPCCR:9a052210
GD25Q256CYIG 00c84019
[0.0000] xburst rtos @ Dec 27 2019 11:48:11, epc: 81f64644
[0.4177] vendor init...
[0.4198] hello world..
[0.4219] mul = 12
[0.4235] CONFIG_VENDOR_NUM = 5
[0.4263] CONFIG_VENDOR_STRING = hello_everyone
```

如果需要开启或者修改宏控的值，在 IconfigTool 工具界，点击进入用户添加的选项



此时可以依据需求修改，修改完成后点击左上角的file、save



修改之后，回到 `freertos` 目录下 `make xxx_defconfig`，再进行编译，结果如下

```
CPM_CPMPCR:98000083
CPM_CPCCR:9a052210
GD25Q256CYIG 00c84019
[0.0000] xburst rtos @ Dec 27 2019 11:55:03, epc: 81f64650
[0.4160] vendor init...
[0.4181] hello world..
[0.4202] 2 + 3 = 5
[0.4219] CONFIG_VENDOR_NUM = 15
[0.4248] CONFIG_VENDOR_STRING = hello_world
```

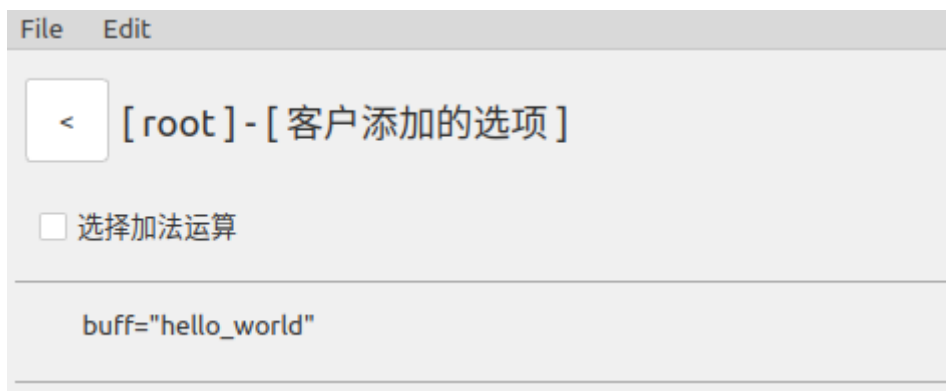
7 用宏来控制文件是否加入编译流程

前文描述的步骤都是将文件加入了编译流程。但可以通过宏来控制我们需要的文件来加入编译流程。以 `add.c` 文件为例,如下图 `Makefile` 文件:

```
1 src-y += vendor.c
2 src-$(CONFIG_VENDOR_ADD) += add.c
3
```

`vendor.c` 如下图，与之前的程序一样。

```
void vendor_init(void)
{
    int b = CONFIG_VENDOR_NUM;
    char buf[20] = CONFIG_VENDOR_STRING;
    printf("vendor init...\n");
    printf("hello world..\n");
    #ifdef CONFIG_VENDOR_ADD
    add(2,3);
    #else
    printf("mul = %d\n",mul(3,4));
    #endif
    printf("CONFIG_VENDOR_NUM = %d\n",b);
    printf("CONFIG_VENDOR_STRING = %s\n",buf);
}
```



IconfigTool 的配置界面如上图所示，未勾选“选择加法运算”，编译如下，没有编译 `add.c`

```
..include package/vendor/vendor.mk
..include package/newlib/newlib.mk
lds.in    build/build.lds.in
CC        vendor/vendor.c
LD        vendor/builtin.o
CC        drivers/clk.c
CC        drivers/console.c
CC        drivers/sfc.c
CC        drivers/uart_console.c
```

烧录结果如下图

```
GD25Q256CYIG 00c84019
[0.0000] xburst rtos @ Dec 27 2019 13:34:56, epc: 81f64644
[0.4168] vendor init...
[0.4189] hello world..
[0.4209] mul = 12
[0.4226] CONFIG_VENDOR_NUM = 15
[0.4254] CONFIG_VENDOR_STRING = hello_world
```

若勾选了“选择加法运算”，编译如下，编译加入 `add.c`



```

CC      drivers/gpio.c
CC      lib/assert.c
CC      drivers/uart_console.c
CC      drivers/console.c
CC      vendor/vendor.c
CC      vendor/add.c
CC      shell/cmds/cmds_ls.c
CC      shell/cmds/cmds_cp.c
CC      shell/cmds/cmds_cat.c
CC      shell/shell_cmd.c

```

烧录结果如下

```

[0.0000] xburst rtos @ Dec 27 2019 13:45:05, epc: 81f64644
[0.4179] vendor init...
[0.4200] hello world..
[0.4221] 2 + 3 = 5
[0.4238] CONFIG_VENDOR_NUM = 15
[0.4267] CONFIG_VENDOR_STRING = hello_world
[0.4306]

```

8 注意事项

与驱动相关文件推荐使用源码编译，仅推荐算法使用.o/.a，与驱动相关的文件最好使用源码编译。

例1:对于不同的板极，soc中.h的内容是不同的，以 soc-x1000 中的 soc/gpio.h 和 soc-x1520 中的 soc/gpio.h，如下图

soc-x1000 中的 soc/gpio.h

```

19  ...GPIO_PULL_HIZ = 0x80, ...//no pull
20  ...GPIO_PULL..... = 0xa0, ...//pull, x1000只支持pull,是什么类型的pull 要看芯片手册
21  ...GPIO_PULL_UP... = 0xa0, ...//pull high
22  ...GPIO_PULL_DOWN = 0xa0, ...//pull down
23  };

```

soc-x1520 中的 soc/gpio.h

```

17  ...GPIO_PULL_HIZ = 0x80, ...//no pull
18  ...GPIO_PULL_UP... = 0xa0, ...//pull high
19  ...GPIO_PULL_DOWN = 0xc0, ...//pull low
20  ...GPIO_PULL_BUSHOLD = 0xe0,
21

```

从以上两张图中明显可以看出，不同的板极引脚功能是不同的。

例2: .o/.a的生成链接了某些驱动，但是当固件更新时，对应的驱动的结构体大小或者接口发生变化的时候也会出错。

所以仅推荐算法使用的.o/.a，与驱动相关的文件最好使用源码编译。

9 相关文件介绍

9.1 package/vendor/vendor.mk

文件中可以使用如下变量执行相应的功能

```
1 package_name          #定义包名,用于 package_depends 依赖时寻找
2
3 package_depends        #定义依赖的包,依赖的包先编译
4
5 package_builtin_src     #定需要编译的文件目录,在改目录中编写Makefile文件控制哪
6
7 package_make_hook
8
9 package_init_hook
10
11 package_finalize_hook
12
13 package_clean_hook
14
15 hook 的执行顺序如下所示: init_hook -> builtin_src -> make_hoook -> rtos镜像 ->
    finalize_hook
```

例如, 在 package/vendor/vendor.mk 中添加

```
1 define vendor_make_hook
2     $(Q)echo "生成.bin文件之前执行"
3 endef
4 define vendor_init_hook
5     $(Q)echo "在编译流程开始前执行"
6 endef
7 define vendor_finalize_hook
8     $(Q)echo "在编译流程结束后执行"
9 endef
10 define vendor_clean_hook
11     $(Q)echo "在 make clean的时候执行"
12 endef
13 package_make_hook = vendor_make_hook
14 package_init_hook = vendor_init_hook
15 package_clean_hook = vendor_clean_hook
16 package_finalize_hook = vendor_finalize_hook
```

9.2 xburst/init.c

`c_main` 函数是工程的入口, 在文件 `init.c` 中, 该文件在 `freertos/xburst/` 目录下。

`c_main` 中的宏控是开启开发板的部分功能, 例如开机时显示编译的时间、cache驱动(cache相关操作)、IRQ驱动(中断)、CLK驱动(时钟)、高精度定时器等, 依据需求用户可以在 `IconfigTool` 中开启。直至 `CONFIG_OS` 前都是相应的宏控。在 `CONFIG_OS` 定义后创建线程 `main_init_thread`, 其中第三方编写的内容函数 `vendor_init` 就在该线程的末尾执行。